

<PROJECT PDF DUMP>

Generated at: 2026-04-23T13:51:15.907350 UTC

Root: C:\Users\morty\video-exchange-bot

Files included: 16

```
=====
FILE: .env
=====
```

```
BOT_TOKEN=8747618457:AAHz0LNqPr4wnSW0dDhT0xLwyREc-JKHINI
ADMINS=5272076117
WEBHOOK_BASE=https://placeholder.onrender.com
WEBHOOK_PATH=/webhook
DATABASE_URL=sqlite+aiosqlite:///bot.db
PORT=10000
```

```
=====
FILE: .env.example
=====
```

```
BOT_TOKEN=123456:ABC-DEF
ADMINS=123456789
WEBHOOK_BASE=https://your-app.onrender.com
WEBHOOK_PATH=/webhook
DATABASE_URL=postgresql+asyncpg://user:password@host:5432/dbname
```

```
=====
FILE: app\__init__.py
=====
```

```
=====
FILE: app\admin_handlers.py
=====
```

```
from datetime import datetime
from decimal import Decimal
from aiogram import Router, F
from aiogram.filters import Command
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.context import FSMContext
from aiogram.types import Message, CallbackQuery, InlineKeyboardMarkup, InlineKeyboardButton
from sqlalchemy import select
from app.config import ADMINS
from app.db import async_session
from app.models import User
from app.services import (
    get_user,
    get_user_by_id,
    get_user_by_username,
    get_user_dossier,
    update_user_balance,
    set_user_ban_status,
    count_pending_videos,
    count_approved_videos,
    count_rejected_videos,
)

router = Router()

# =====
# STATES
# =====
class AdminUserState(StatesGroup):
    waiting_user_id = State()
    waiting_coins_amount = State()
    waiting_message_text = State()

class AdminManageState(StatesGroup):
    waiting_new_admin = State()
    waiting_remove_admin = State()
```

```

# =====
# HELPERS
# =====
def is_super_admin(tid: int) -> bool:
    return tid in ADMINS

# BUG FIX #1: check_admin async, await cmd_admin_redirect
# user_handlers.py. async - await
async def check_admin(tid: int) -> bool:
    if tid in ADMINS:
        return True
    async with async_session() as session:
        user = await get_user(session, tid)
        if user and user.is_admin:
            return True
    return False

# =====
# ADMIN PANEL ENTRY
# =====
@router.message(Command("admin"))
async def cmd_admin(message: Message):
    if not await check_admin(message.from_user.id):
        return
    sa = is_super_admin(message.from_user.id)
    buttons = [
        InlineKeyboardButton(text="■", callback_data="admin_queue_info"),
        InlineKeyboardButton(text="■", callback_data="admin_manage_users"),
    ]
    if sa:
        buttons.append(
            InlineKeyboardButton(text="■", callback_data="admin_manage_admins")
        )
    kb = InlineKeyboardMarkup(inline_keyboard=buttons)
    await message.answer("■ <b>■■■■■-■■■■■</b>", parse_mode="HTML", reply_markup=kb)

# =====
# QUEUE INFO
# =====
@router.callback_query(F.data == "admin_queue_info")
async def cb_queue(callback: CallbackQuery):
    if not await check_admin(callback.from_user.id):
        return
    async with async_session() as session:
        p = await count_pending_videos(session)
        a = await count_approved_videos(session)
        r = await count_rejected_videos(session)
    text = (
        f"■ <b>■■■■■■■■■■■■■■■■■■■■</b>\n\n"
        f"■ ■■■■■■■■■■■■■■■■■■■■■: <b>{p}</b>\n"
        f"■ ■■■■■■■■■■■■■■■■■■■■■: <b>{a}</b>\n"
        f"■ ■■■■■■■■■■■■■■■■■■■■■: <b>{r}</b>"
    )
    await callback.message.edit_text(text, parse_mode="HTML")
    await callback.answer()

# =====
# ADMIN MANAGEMENT SYSTEM
# =====
@router.callback_query(F.data == "admin_manage_admins")
async def manage_admins_menu(callback: CallbackQuery):
    if not is_super_admin(callback.from_user.id):

```

```

        return
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■", callback_data="admin_list_admins")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_add_admin")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_remove_admin")],
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="admin_back")],
    ])
    await callback.message.edit_text(
        "■ <b>■■■■■■■■■■ ■■■■■■■■</b>",
        parse_mode="HTML",
        reply_markup=kb
    )
    await callback.answer()

@router.callback_query(F.data == "admin_back")
async def admin_back(callback: CallbackQuery):
    # BUG FIX #2: cmd_admin Message, ■ ■■ CallbackQuery.
    # ■■■■■ ■■■■■■■■■■ callback.message, ■ ■■ ■■■■ callback.
    await cmd_admin(callback.message)
    await callback.answer()

# LIST ADMINS
@router.callback_query(F.data == "admin_list_admins")
async def list_admins(callback: CallbackQuery):
    if not is_super_admin(callback.from_user.id):
        return
    async with async_session() as session:
        result = await session.execute(
            select(User).where(User.is_admin == True)
        )
        admins = result.scalars().all()
    if not admins:
        text = "■■■■■■■ ■■■."
    else:
        text = "■ <b>■■■■■■ ■■■■■■■■</b>\n\n"
        for admin in admins:
            text += f"• {admin.first_name or '■■■ ■■■■■'} (ID: <code>{admin.telegram_id}</code>
>)\n"
        await callback.message.answer(text, parse_mode="HTML")
        await callback.answer()

# ADD ADMIN
@router.callback_query(F.data == "admin_add_admin")
async def add_admin_start(callback: CallbackQuery, state: FSMContext):
    if not is_super_admin(callback.from_user.id):
        return
    await state.set_state(AdminManageState.waiting_new_admin)
    await callback.message.answer(
        "■■■■■■■ Telegram ID ■■■■■■■■■■■■ ■■■ ■■■■■■■■■■ ■ ■■■■■■■■:"
    )
    await callback.answer()

@router.message(AdminManageState.waiting_new_admin)
async def process_add_admin(message: Message, state: FSMContext):
    if not is_super_admin(message.from_user.id):
        return
    if not message.text.isdigit():
        await message.answer("■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■ Telegram ID.")
        return
    telegram_id = int(message.text)
    async with async_session() as session:
        user = await get_user(session, telegram_id)
        if not user:
            await message.answer(
                "■■■■■■■■■■■■■■■ ■■ ■■■■■■■■.\n"
                "■■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ /start."
            )

```

```

    )
    await state.clear()
    return
    user.is_admin = True
    await session.commit()
    await message.answer(f"■ ■■■■■■■■■■■■ {telegram_id} ■■■■■■■■ ■■■■■■■■.")
    await state.clear()

# REMOVE ADMIN
@router.callback_query(F.data == "admin_remove_admin")
async def remove_admin_start(callback: CallbackQuery, state: FSMContext):
    if not is_super_admin(callback.from_user.id):
        return
    await state.set_state(AdminManageState.waiting_remove_admin)
    await callback.message.answer(
        "■■■■■■■ Telegram ID ■■■■■■ ■■■ ■■■■■■■■■■:"
    )
    await callback.answer()

@router.message(AdminManageState.waiting_remove_admin)
async def process_remove_admin(message: Message, state: FSMContext):
    if not is_super_admin(message.from_user.id):
        return
    if not message.text.isdigit():
        await message.answer("■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■ Telegram ID.")
        return
    telegram_id = int(message.text)
    async with async_session() as session:
        user = await get_user(session, telegram_id)
        if not user or not user.is_admin:
            await message.answer("■■■■ ■■■■■■■■■■■■■■ ■■ ■■■■■■■■■ ■■■■■■■■.")
            await state.clear()
            return
        user.is_admin = False
        await session.commit()
    await message.answer(f"■ ■■■■■■■■■■■■■■ {telegram_id} ■■■■■■■■ ■■ ■■■■■■■■.")
    await state.clear()

```

```

        return float(raw)
    except Exception:
        return default

BOT_TOKEN = _get_str("BOT_TOKEN")
DATABASE_URL = _get_str("DATABASE_URL")
BOT_USERNAME = _get_str("BOT_USERNAME")
PROVIDER_TOKEN = _get_str("PROVIDER_TOKEN")
WEBHOOK_BASE = _get_str("WEBHOOK_BASE")
WEBHOOK_PATH = _get_str("WEBHOOK_PATH")
LOG_CHAT_ID = _get_str("LOG_CHAT_ID")

ADMINS_RAW = _get_str("ADMINS")
ADMINS = []
for x in ADMINS_RAW.split(","):
    x = x.strip()
    if x.isdigit():
        ADMINS.append(int(x))

PORT = _get_int("PORT", 10000)

STARTING_BALANCE = _get_float("STARTING_BALANCE", 2.0)
WATCH_COST = _get_float("WATCH_COST", 1.0)
UPLOAD_REWARD = _get_float("UPLOAD_REWARD", 0.5)

REFERRAL_REWARD_INVITER = _get_float("REFERRAL_REWARD_INVITER", 2.0)
REFERRAL_REWARD_NEW_USER = _get_float("REFERRAL_REWARD_NEW_USER", 1.0)

STARS_TO_COINS_RATE = _get_float("STARS_TO_COINS_RATE", 2.0)

STARS_PACKAGES = {
    "pack_1": {"stars": 1, "coins": 2, "title": "2 \u043c\u043e\u043d\u0435\u0442\u044b"},
    "pack_5": {"stars": 5, "coins": 10, "title": "10 \u043c\u043e\u043d\u0435\u0442"},
    "pack_10": {"stars": 10, "coins": 20, "title": "20 \u043c\u043e\u043d\u0435\u0442"},
    "pack_25": {"stars": 25, "coins": 50, "title": "50 \u043c\u043e\u043d\u0435\u0442"},
    "pack_50": {"stars": 50, "coins": 100, "title": "100 \u043c\u043e\u043d\u0435\u0442"},
}

MONEY_PACKAGES = {
    "money_99": {"amount": 9900, "coins": 250, "title": "250 \u043c\u043e\u043d\u0435\u0442"},
    "money_199": {"amount": 19900, "coins": 600, "title": "600 \u043c\u043e\u043d\u0435\u0442"},
    "money_499": {"amount": 49900, "coins": 1800, "title": "1800 \u043c\u043e\u043d\u0435\u0442"},
},
}

OFFER_BROADCAST_INTERVAL_HOURS = _get_float("OFFER_BROADCAST_INTERVAL_HOURS", 2.5)

# === LEVELS ===
LEVEL_XP_BASE = _get_int("LEVEL_XP_BASE", 100)
LEVEL_XP_MULTIPLIER = _get_float("LEVEL_XP_MULTIPLIER", 1.5)

XP_PER_WATCH = _get_int("XP_PER_WATCH", 5)
XP_PER_UPLOAD = _get_int("XP_PER_UPLOAD", 20)
XP_PER_RATING = _get_int("XP_PER_RATING", 2)
XP_PER_COMMENT = _get_int("XP_PER_COMMENT", 3)
XP_PER_REACTION = _get_int("XP_PER_REACTION", 1)
XP_PER_GAME = _get_int("XP_PER_GAME", 3)

# === VIP ===
VIP_PRICE_STARS = _get_int("VIP_PRICE_STARS", 50)
VIP_DURATION_DAYS = _get_int("VIP_DURATION_DAYS", 30)
VIP_BONUS_MULTIPLIER = _get_float("VIP_BONUS_MULTIPLIER", 3.0)
VIP_FREE_PHOTOS = True
VIP_WATCH_DISCOUNT = _get_float("VIP_WATCH_DISCOUNT", 0.5)

# === GAMES ===
DICE_MIN_BET = _get_int("DICE_MIN_BET", 1)
DICE_MAX_BET = _get_int("DICE_MAX_BET", 50)

```

```

# === QUESTS ===
DAILY_QUESTS = [
    {"type": "watch", "target": 3, "reward": 2, "desc": "\u0041\u0043\u0044\u0043\u0043\u0042\u0044\u0043 3 \u0043\u0043\u0043\u0043\u0043\u0043"},
    {"type": "upload", "target": 1, "reward": 3, "desc": "\u0041\u0043\u0043\u0044\u0043\u0043\u0043\u0043 1 \u0043\u0043\u0043\u0043\u0043\u0043\u0043\u0043"},
    {"type": "rate", "target": 3, "reward": 1, "desc": "\u0041\u0044\u0043\u0043\u0043\u0043 3 \u0043\u0043\u0043\u0043\u0043\u0043"},
    {"type": "comment", "target": 2, "reward": 2, "desc": "\u0041\u0044\u0042\u0043\u0043\u0044\u0043\u0043\u0043\u0043\u0043\u0043"},
    {"type": "react", "target": 5, "reward": 1, "desc": "\u0041\u0043\u0044\u0042\u0043\u0043\u0043\u0043\u0043\u0043\u0043\u0043"},
]

PREMIUM_DAILY_QUESTS = [
    {"type": "watch", "target": 10, "reward": 8, "desc": "VIP: \u0041\u0043\u0044\u0043\u0043\u0043\u0044\u0040\u0043 10 \u0043\u0043\u0043\u0043\u0043\u0043"},
    {"type": "comment", "target": 5, "reward": 5, "desc": "VIP: \u0041\u0044\u0042\u0043\u0043\u0044\u0043\u0043\u0043\u0043\u0043\u0043"},
]

REACTION_TYPES = ["\u0001f525", "\u02764\u0fe0f", "\u0001f602", "\u0001f44d", "\u0001f4af"]

# === ANTI-SPAM COMMENTS ===
COMMENTS_PER_10_MIN = _get_int("COMMENTS_PER_10_MIN", 5)
COMMENT_MIN_INTERVAL_SEC = _get_int("COMMENT_MIN_INTERVAL_SEC", 15)

# === WEEKLY REWARDS ===
WEEKLY_TOP1_REWARD = _get_float("WEEKLY_TOP1_REWARD", 25.0)
WEEKLY_TOP2_REWARD = _get_float("WEEKLY_TOP2_REWARD", 15.0)
WEEKLY_TOP3_REWARD = _get_float("WEEKLY_TOP3_REWARD", 10.0)

# === MONETIZATION ===
PIN_OFFER_COST = _get_float("PIN_OFFER_COST", 100.0)
BUMP_VIDEO_COST = _get_float("BUMP_VIDEO_COST", 25.0)

=====
FILE: app\db.py
=====

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession

from app.config import DATABASE_URL
from app.models import Base

def _fix_url(url: str) -> str:
    if url.startswith("postgres://"):
        return url.replace("postgres://", "postgresql+asyncpg://", 1)
    if url.startswith("postgresql://"):
        return url.replace("postgresql://", "postgresql+asyncpg://", 1)
    return url

engine = create_async_engine(
    _fix_url(DATABASE_URL),
    echo=False,
)

async_session = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

```

```

async def reset_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
        await conn.run_sync(Base.metadata.create_all)

=====
FILE: app\keyboards.py
=====
from aiogram.types import InlineKeyboardMarkup, InlineKeyboardButton, ReplyKeyboardMarkup, KeyboardButton

# Buttons
BTN_WATCH = "■ ■■■■■■■■"
BTN_UPLOAD = "■ ■■■■■■■■"
BTN_PROFILE = "■ ■■■■■■■■"
BTN_BUY = "■ ■■■■■■■■"
BTN_OFFERS = "■ ■■■■■■■■"
BTN_REFERRALS = "■ ■■■■■■■■"
BTN_BONUS = "■ ■■■■■■■■"
BTN_ADMIN = "■ ■■■■■■■■"
BTN_GAMES = "■ ■■■■■■■■"
BTN_TOPS = "■ ■■■■■■■■"
BTN_QUESTS = "■ ■■■■■■■■"
BTN_VIP = "■ VIP"
BTN_LEVEL = "■ ■■■■■■■■"

def main_menu(is_admin: bool = False) -> ReplyKeyboardMarkup:
    kb = [
        [KeyboardButton(text=BTN_WATCH), KeyboardButton(text=BTN_UPLOAD)],
        [KeyboardButton(text=BTN_PROFILE), KeyboardButton(text=BTN_GAMES)],
        [KeyboardButton(text=BTN_OFFERS), KeyboardButton(text=BTN_BUY)],
        [KeyboardButton(text=BTN_REFERRALS), KeyboardButton(text=BTN_BONUS)],
        [KeyboardButton(text=BTN_QUESTS), KeyboardButton(text=BTN_TOPS)],
        [KeyboardButton(text=BTN_LEVEL), KeyboardButton(text=BTN_VIP)],
    ]
    if is_admin:
        kb.append([KeyboardButton(text=BTN_ADMIN)])
    return ReplyKeyboardMarkup(keyboard=kb, resize_keyboard=True)

def admin_center_keyboard(is_super_admin: bool = False) -> InlineKeyboardMarkup:
    buttons = [
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_queue_info")],
        [InlineKeyboardButton(text="■ ■■■■■■■■+", callback_data="admin_extended_stats")],
        [InlineKeyboardButton(text="■ ■■■■■■■■ ■■■■■■■■■■", callback_data="admin_manage_users")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_get_pending")],
        [InlineKeyboardButton(text="■ ■■■■■■■■ ■■■", callback_data="admin_approve_all")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_offers_menu")],
    ]
    if is_super_admin:
        buttons.append([InlineKeyboardButton(text="■ ■■■■■■■■ ■■■■■■■■■■", callback_data="admin_manage_admins")])
    return InlineKeyboardMarkup(inline_keyboard=buttons)

def moderation_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="■ ■■■■■■■■", callback_data=f"mod_approve:{video_id}"),
            InlineKeyboardButton(text="■ ■■■■■■■■", callback_data=f"mod_reject:{video_id}"),
        ],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="admin_get_pending")],
    ])

def rejection_reason_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data=f"reject_reason:{video_id}:duplicate")],
        [InlineKeyboardButton(text="■ ■■ ■■ ■■■■■■■■■■", callback_data=f"reject_reason:{video_id}:off_topic")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data=f"reject_reason:{video_id}:other")],
    ])

```

```

],
])

def admin_after_action_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■■■■■", callback_data="admin_get_pending")],
        [InlineKeyboardButton(text="■ ■■■■■-■■■■■", callback_data="admin_center")],
    ])

def offer_view_keyboard(offer_id: int, channel_url: str) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■■■ ■ ■■■■■", url=channel_url)],
        [InlineKeyboardButton(text="■ ■■■■■■", callback_data=f"offer_start:{offer_id}")],
        [InlineKeyboardButton(text="■ ■■■■■■■■■■", callback_data=f"offer_check:{offer_id}")],
    ])

def games_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="game_dice")],
        [InlineKeyboardButton(text="■ ■■■■■■■", callback_data="game_coinflip")],
        [InlineKeyboardButton(text="■ ■■■■■■ ■■■■■", callback_data="game_guess")],
    ])

def tops_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■ ■■■■■■■■■■■", callback_data="top_uploaders")],
        [InlineKeyboardButton(text="■ ■■■ ■■■■■■■■■", callback_data="top_viewers")],
        [InlineKeyboardButton(text="■ ■■■ ■■ XP", callback_data="top_levels")],
        [InlineKeyboardButton(text="■ ■■■ ■■■■■■■", callback_data="top_richest")],
    ])

def watch_choice_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■", callback_data="watch_video_content")],
        [InlineKeyboardButton(text="■ ■■■■", callback_data="watch_photo_content")],
    ])

def video_rating_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="■ 1", callback_data=f"rate:{video_id}:1"),
            InlineKeyboardButton(text="■ 2", callback_data=f"rate:{video_id}:2"),
            InlineKeyboardButton(text="■ 3", callback_data=f"rate:{video_id}:3"),
            InlineKeyboardButton(text="■ 4", callback_data=f"rate:{video_id}:4"),
            InlineKeyboardButton(text="■ 5", callback_data=f"rate:{video_id}:5"),
        ],
        [InlineKeyboardButton(text="■ ■■■■■■■", callback_data=f"comments:{video_id}")],
        [InlineKeyboardButton(text="■ ■■■■■■■■■", callback_data="watch_next")],
    ])

def photo_actions_keyboard(photo_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■■■■ ■■■■", callback_data="watch_next_photo")],
    ])

def rules_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ ■■■■■■■■ ■■■■■■■", callback_data="accept_rules")],
    ])

def buy_coins_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="■ 10 ■■■■■ (5 Stars)", callback_data="buy:pack_5")],
        [InlineKeyboardButton(text="■ 20 ■■■■■ (10 Stars)", callback_data="buy:pack_10")],
        [InlineKeyboardButton(text="■ 50 ■■■■■ (25 Stars)", callback_data="buy:pack_25")],
        [InlineKeyboardButton(text="■ 100 ■■■■■ (50 Stars)", callback_data="buy:pack_50")],
        [InlineKeyboardButton(text="■ ■■■■ ■■■■■", callback_data="buy_custom")],
    ])

def vip_buy_keyboard() -> InlineKeyboardMarkup:

```



```

return InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(text="■ ■■■■■■ VIP", callback_data="buy_vip")],
])

def offers_list_keyboard(offers) -> InlineKeyboardMarkup:
    buttons = []
    for o in offers:
        icon = "■" if o.is_active else "■"
        buttons.append([InlineKeyboardButton(text=f"{icon} {o.title}", callback_data=f"offer_open:{o.id}"))]
    return InlineKeyboardMarkup(inline_keyboard=buttons)

def quests_keyboard(quests) -> InlineKeyboardMarkup:
    buttons = []
    for q in quests:
        status = "■" if q.completed else "■"
        buttons.append([InlineKeyboardButton(text=f"{status} {q.quest_type}: {q.progress}/{q.target}", callback_data=f"quest_claim:{q.id}"))]
    return InlineKeyboardMarkup(inline_keyboard=buttons)

def reaction_menu_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
            InlineKeyboardButton(text="♥■", callback_data=f"react:{video_id}:♥■"),
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
            InlineKeyboardButton(text="■", callback_data=f"react:{video_id}:■"),
        ]
    ])

```

```

=====
FILE: app\logger.py
=====

```

```

import json
import logging
import traceback
from datetime import datetime

```

```

def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)

```

```

def _make_payload(message: str, **kwargs) -> str:
    payload = {
        "time": datetime.utcnow().isoformat() + "Z",
        "message": message,
    }
    payload.update(kwargs)
    return json.dumps(payload, ensure_ascii=False)

```

```

def setup_logging(level: int = logging.INFO):
    root = logging.getLogger()
    root.setLevel(level)

    while root.handlers:
        root.handlers.pop()

    handler = logging.StreamHandler()
    handler.setLevel(level)
    handler.setFormatter(logging.Formatter("%(message)s"))
    root.addHandler(handler)

```

```

def log_info(logger: logging.Logger, message: str, **kwargs):
    logger.info(_make_payload(message, **kwargs))

```

```

def log_warning(logger: logging.Logger, message: str, **kwargs):
    logger.warning(_make_payload(message, **kwargs))

def log_error(logger: logging.Logger, message: str, **kwargs):
    logger.error(_make_payload(message, **kwargs))

def log_exception(logger: logging.Logger, message: str, **kwargs):
    kwargs["traceback"] = traceback.format_exc()
    logger.error(_make_payload(message, **kwargs))

def log_event(logger: logging.Logger, level: int, message: str, **kwargs):
    logger.log(level, _make_payload(message, **kwargs))

=====
FILE: app\main.py
=====
import asyncio
import logging
from aiohttp import web
from aiogram import Bot, Dispatcher
from aiogram.client.default import DefaultBotProperties
from aiogram.enums import ParseMode
from app.config import BOT_TOKEN, PORT
from app.db import engine, async_session, init_db
from app.user_handlers import router as user_router
from app.admin_handlers import router as admin_router
from app.logger import setup_logging, get_logger, log_info

setup_logging()
logger = get_logger(__name__)

async def on_startup(app):
    from app.migrate import main as run_migrations
    await init_db()
    try:
        await run_migrations()
    except Exception as e:
        log_info(logger, f"Migrations warning: {e}")
    log_info(logger, "Bot starting up... DB initialized")

async def main():
    if not BOT_TOKEN:
        raise RuntimeError("BOT_TOKEN is empty")

    bot = Bot(
        token=BOT_TOKEN,
        default=DefaultBotProperties(parse_mode=ParseMode.HTML),
    )

    dp = Dispatcher()
    dp.include_router(user_router)
    dp.include_router(admin_router)

    app = web.Application()
    app.on_startup.append(on_startup)

    runner = web.AppRunner(app)
    await runner.setup()
    site = web.TCPSite(runner, "0.0.0.0", int(PORT or 10000))
    await site.start()

    try:
        log_info(logger, "Polling started")
        await dp.start_polling(bot)
    finally:

```

```
await runner.cleanup()
await bot.session.close()
await engine.dispose()
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

```
=====
FILE: app\make_project_pdf.py
=====
```

```
import os
import logging
from pathlib import Path
from datetime import datetime

from reportlab.lib.pagesizes import A4
from reportlab.pdfbase.pdfmetrics import stringWidth
from reportlab.pdfgen import canvas

import sys
sys.path.insert(0, str(Path(__file__).parent.parent))
from app.logger import setup_logging, get_logger, log_info, log_warning, log_exception
from pathlib import Path
```

```
PROJECT_ROOT = Path(".")
OUTPUT_FILE = "project_dump.pdf"
```

```
MAX_PAGES = 90
MAX_SIZE_MB = 15
```

```
INCLUDE_EXTENSIONS = {
    ".py",
    ".txt",
    ".md",
    ".json",
    ".yaml",
    ".yml",
    ".toml",
    ".ini",
    ".env",
    ".sql",
}
```

```
SPECIAL_FILENAMES = {
    "Dockerfile",
    "Procfile",
    "requirements.txt",
    "runtime.txt",
    ".env",
}
```

```
EXCLUDED_DIRS = {
    ".git",
    ".venv",
    "venv",
    "__pycache__",
    "node_modules",
    ".idea",
    ".vscode",
    "dist",
    "build",
    ".pytest_cache",
    ".mypy_cache",
    ".ruff_cache",
    ".cache",
}
```

```
EXCLUDED_FILES = {
```

```

    OUTPUT_FILE,
}

MAX_FILE_SIZE_KB = 300

def is_text_candidate(path: Path) -> bool:
    if path.name in SPECIAL_FILENAMES:
        return True
    if path.suffix.lower() in INCLUDE_EXTENSIONS:
        return True
    if path.name.startswith(".env"):
        return True
    return False

def should_skip(path: Path) -> bool:
    if path.is_dir():
        return True

    if path.name in EXCLUDED_FILES:
        return True

    parts = set(path.parts)
    if parts & EXCLUDED_DIRS:
        return True

    if not is_text_candidate(path):
        return True

    try:
        if path.stat().st_size > MAX_FILE_SIZE_KB * 1024:
            return True
    except Exception:
        return True

    return False

def iter_project_files(root: Path):
    files = []
    for path in root.rglob("*"):
        if should_skip(path):
            continue
        files.append(path)
    files.sort(key=lambda p: str(p).lower())
    return files

def safe_read_text(path: Path) -> str:
    for enc in ("utf-8", "utf-8-sig", "cp1251", "latin-1"):
        try:
            return path.read_text(encoding=enc)
        except Exception:
            pass
    return "[[FAILED TO READ FILE]]"

def wrap_line(line: str, font_name: str, font_size: int, max_width: float):
    if not line:
        return [""]

    if stringWidth(line, font_name, font_size) <= max_width:
        return [line]

    parts = []
    current = ""

    for ch in line:
        test = current + ch

```

```

        if stringWidth(test, font_name, font_size) <= max_width:
            current = test
        else:
            if current:
                parts.append(current)
            current = ch

    if current:
        parts.append(current)

    return parts

def build_blocks(root: Path):
    files = iter_project_files(root)
    blocks = []

    header = [
        "<PROJECT PDF DUMP>",
        f"Generated at: {datetime.utcnow().isoformat()} UTC",
        f"Root: {root.resolve()}",
        f"Files included: {len(files)}",
        ""
    ]
    blocks.append("\n".join(header))

    for path in files:
        rel = path.relative_to(root)
        content = safe_read_text(path).rstrip()

        block = [
            "=" * 80,
            f"FILE: {rel}",
            "=" * 80,
            content,
            ""
        ]
        blocks.append("\n".join(block))

    return blocks, files

def render_pdf(blocks, output_path: str):
    page_width, page_height = A4

    configs = [
        {"font_size": 9, "line_gap": 11, "margin": 36},
        {"font_size": 8, "line_gap": 9, "margin": 26},
        {"font_size": 7, "line_gap": 8, "margin": 18},
        {"font_size": 6, "line_gap": 7, "margin": 14},
    ]

    best_result = None

    for cfg in configs:
        c = canvas.Canvas(output_path, pagesize=A4)
        font_name = "Courier"
        font_size = cfg["font_size"]
        line_gap = cfg["line_gap"]
        margin = cfg["margin"]

        usable_width = page_width - margin * 2
        y = page_height - margin
        pages = 1

        c.setFont(font_name, font_size)

        def new_page():
            nonlocal y, pages
            c.showPage()

```

```

        c.setFont(font_name, font_size)
        y = page_height - margin
        pages += 1

    for block in blocks:
        for raw_line in block.splitlines():
            wrapped = wrap_line(raw_line, font_name, font_size, usable_width)
            for line in wrapped:
                if y < margin:
                    new_page()
                c.drawString(margin, y, line)
                y -= line_gap

            if y < margin:
                new_page()
            y -= line_gap

    c.save()

    size_bytes = os.path.getsize(output_path)
    size_mb = size_bytes / (1024 * 1024)

    best_result = {
        "pages": pages,
        "size_bytes": size_bytes,
        "size_mb": size_mb,
        "config": cfg,
    }

    if pages <= MAX_PAGES and size_mb <= MAX_SIZE_MB:
        return best_result

    return best_result

def main():
    setup_logging()
    logger = get_logger(__name__)

    try:
        blocks, files = build_blocks(PROJECT_ROOT)
        result = render_pdf(blocks, OUTPUT_FILE)

        logger.info("=" * 60)
        logger.info(f"PDF created: {OUTPUT_FILE}")
        logger.info(f"Files included: {len(files)}")
        logger.info(f"Pages: {result['pages']}")
        logger.info(f"Size: {result['size_mb']:.2f} MB")
        logger.info(f"Config used: {result['config']}")
        logger.info("=" * 60)

        if result["pages"] > MAX_PAGES:
            logger.warning(f"PDF exceeds page limit ({result['pages']} > {MAX_PAGES})")

        if result["size_mb"] > MAX_SIZE_MB:
            logger.warning(f"PDF exceeds size limit ({result['size_mb']:.2f} MB > {MAX_SIZE_MB} MB)")

    except Exception:
        log_exception(logger, "Error during PDF generation")

if __name__ == "__main__":
    main()

=====
FILE: app\migrate.py
=====
import asyncio
import sys
sys.path.insert(0, '..')

```

```
from app.logger import setup_logging, get_logger, log_info
from sqlalchemy import text
from app.db import engine
```

```
setup_logging()
```

```
MIGRATIONS = [
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS created_by_user_id INTEGER NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS offer_kind VARCHAR(30) DEFAULT 'system';
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promotion_tier VARCHAR(30) DEFAULT 'basic';
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS payment_method VARCHAR(20) NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_coins NUMERIC(12,2) DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_rub INTEGER DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_stars INTEGER DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS priority_level INTEGER DEFAULT 10;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS is_featured BOOLEAN DEFAULT FALSE;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS moderation_comment TEXT NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS approved_at TIMESTAMPTZ NULL;
    """,
    """
    CREATE INDEX IF NOT EXISTS ix_offers_created_by_user_id ON offers (created_by_user_id);
    """,
    """
    CREATE INDEX IF NOT EXISTS ix_offers_status ON offers (status);
    """,
    """
    CREATE TABLE IF NOT EXISTS game_sessions (
        id SERIAL PRIMARY KEY,
        user_id INTEGER NOT NULL REFERENCES users(id),
        window_start TIMESTAMP NOT NULL,
        games_played INTEGER DEFAULT 0,
        paid_at TIMESTAMP NULL
    );
    """,
    """
    CREATE INDEX IF NOT EXISTS ix_game_sessions_user_id ON game_sessions (user_id);
    """,
]
```

```
logger = get_logger(__name__)
```

```
async def main():
    async with engine.begin() as conn:
        for sql in MIGRATIONS:
            await conn.execute(text(sql))
    await engine.dispose()
    log_info(logger, "Migrations applied successfully")
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

```
=====
FILE: app\models.py
=====
```

```
from datetime import datetime
from decimal import Decimal
```

```
from sqlalchemy import (
    BigInteger,
    String,
    Numeric,
    Boolean,
    Integer,
    DateTime,
    ForeignKey,
    UniqueConstraint,
    Text,
    Date,
)
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
```

```
class Base(DeclarativeBase):
    pass
```

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    telegram_id: Mapped[int] = mapped_column(BigInteger, unique=True, nullable=False, index=True)
)
    username: Mapped[str | None] = mapped_column(String(255), nullable=True, index=True)
    first_name: Mapped[str | None] = mapped_column(String(255), nullable=True)
    last_name: Mapped[str | None] = mapped_column(String(255), nullable=True)
    phone_number: Mapped[str | None] = mapped_column(String(50), nullable=True)
    balance: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    status: Mapped[str] = mapped_column(String(50), default="active")
    is_admin: Mapped[bool] = mapped_column(Boolean, default=False)
    agreed_to_rules: Mapped[bool] = mapped_column(Boolean, default=False)
    referral_code: Mapped[str | None] = mapped_column(String(50), unique=True, nullable=True)
    referred_by_user_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"), nullable=True)
e)
    referral_earnings: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    last_bonus_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    xp: Mapped[int] = mapped_column(Integer, default=0)
    level: Mapped[int] = mapped_column(Integer, default=1)
    vip_until: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    videos: Mapped[list["Video"]] = relationship(back_populates="uploader")
    views: Mapped[list["VideoView"]] = relationship(back_populates="user")
    ratings: Mapped[list["VideoRating"]] = relationship(back_populates="user")
    payments: Mapped[list["Payment"]] = relationship(back_populates="user")
    comments: Mapped[list["Comment"]] = relationship(back_populates="user")
    reactions: Mapped[list["ContentReaction"]] = relationship(back_populates="user")
```



```

game_history: Mapped[list["GameHistory"]] = relationship(back_populates="user")
quest_progress: Mapped[list["DailyQuestProgress"]] = relationship(back_populates="user")
game_sessions: Mapped[list["GameSession"]] = relationship(back_populates="user")
action_logs: Mapped[list["UserActionLog"]] = relationship(back_populates="user")
user_offers: Mapped[list["Offer"]] = relationship(back_populates="creator")

```

```

class UserActionLog(Base):
    __tablename__ = "user_action_logs"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    action: Mapped[str] = mapped_column(String(255), nullable=False)
    details: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="action_logs")

class Video(Base):
    __tablename__ = "videos"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    uploader_user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    content_type: Mapped[str] = mapped_column(String(20), default="video")
    telegram_file_id: Mapped[str] = mapped_column(Text, nullable=False)
    telegram_file_unique_id: Mapped[str] = mapped_column(String(255), nullable=False, index=True)

    status: Mapped[str] = mapped_column(String(20), default="pending")
    duration_seconds: Mapped[int | None] = mapped_column(Integer, nullable=True)
    file_size: Mapped[int | None] = mapped_column(BigInteger, nullable=True)
    rejection_reason: Mapped[str | None] = mapped_column(String(255), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    uploader: Mapped["User"] = relationship(back_populates="videos")
    views: Mapped[list["VideoView"]] = relationship(back_populates="video")
    ratings: Mapped[list["VideoRating"]] = relationship(back_populates="video")
    comments: Mapped[list["Comment"]] = relationship(back_populates="video")
    reactions: Mapped[list["ContentReaction"]] = relationship(back_populates="video")

class VideoView(Base):
    __tablename__ = "video_views"

    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_view"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    watched_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="views")
    video: Mapped["Video"] = relationship(back_populates="views")

class VideoRating(Base):
    __tablename__ = "video_ratings"

    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_rating"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    rating: Mapped[int] = mapped_column(Integer, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

```

```

user: Mapped["User"] = relationship(back_populates="ratings")
video: Mapped["Video"] = relationship(back_populates="ratings")

class Comment(Base):
    __tablename__ = "comments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    text: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="comments")
    video: Mapped["Video"] = relationship(back_populates="comments")

class ContentReaction(Base):
    __tablename__ = "content_reactions"

    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_reaction"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    reaction_type: Mapped[str] = mapped_column(String(10), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="reactions")
    video: Mapped["Video"] = relationship(back_populates="reactions")

class Payment(Base):
    __tablename__ = "payments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    payload: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    stars_amount: Mapped[int] = mapped_column(Integer, nullable=False)
    coins_amount: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="pending")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="payments")

class Offer(Base):
    __tablename__ = "offers"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    creator_user_id: Mapped[int | None] = mapped_column(ForeignKey("users.id"), nullable=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    channel_url: Mapped[str] = mapped_column(Text, nullable=False)
    reward_preview: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=5)
    reward_final: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=35)
    penalty_unsubscribe: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=40)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    status: Mapped[str] = mapped_column(String(20), default="approved")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    creator: Mapped["User"] = relationship(back_populates="user_offers")

class OfferParticipation(Base):
    __tablename__ = "offer_participations"

    __table_args__ = (

```

```

        UniqueConstraint("user_id", "offer_id", name="uq_user_offer"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    offer_id: Mapped[int] = mapped_column(ForeignKey("offers.id"), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="started")
    reward_given: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    checked_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)


class GameHistory(Base):
    __tablename__ = "game_history"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    game_type: Mapped[str] = mapped_column(String(50), nullable=False)
    bet: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    result: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    details: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="game_history")


class DailyQuestProgress(Base):
    __tablename__ = "daily_quest_progress"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    quest_type: Mapped[str] = mapped_column(String(50), nullable=False)
    quest_date: Mapped[datetime] = mapped_column(Date, nullable=False)
    progress: Mapped[int] = mapped_column(Integer, default=0)
    target: Mapped[int] = mapped_column(Integer, nullable=False)
    reward: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    completed: Mapped[bool] = mapped_column(Boolean, default=False)
    reward_claimed: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="quest_progress")


class GameSession(Base):
    __tablename__ = "game_sessions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
    window_start: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    games_played: Mapped[int] = mapped_column(Integer, default=0)
    paid_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    user: Mapped["User"] = relationship(back_populates="game_sessions")


=====
FILE: app\services.py
=====

import uuid
import random
from datetime import datetime, timedelta
from decimal import Decimal, ROUND_DOWN
from sqlalchemy import select, func, desc
from sqlalchemy.ext.asyncio import AsyncSession

from app.models import (
    User,
    Video,
    VideoView,
    VideoRating,
    Payment,

```

```

Offer,
OfferParticipation,
Comment,
ContentReaction,
GameHistory,
DailyQuestProgress,
GameSession,
UserActionLog,
)
from app.config import (
    STARTING_BALANCE,
    WATCH_COST,
    UPLOAD_REWARD,
    REFERRAL_REWARD_INVITER,
    REFERRAL_REWARD_NEW_USER,
    STARS_PACKAGES,
    STARS_TO_COINS_RATE,
    MONEY_PACKAGES,
    WEEKLY_TOP1_REWARD,
    WEEKLY_TOP2_REWARD,
    WEEKLY_TOP3_REWARD,
    DAILY_QUESTS,
    PREMIUM_DAILY_QUESTS,
    BUMP_VIDEO_COST,
    PIN_OFFER_COST,
)

PHOTO_UPLOAD_REWARD = Decimal("0.1")
OFFER_STEP_1_REWARD = Decimal("5")
OFFER_PENALTY = Decimal("40")
GAME_SESSION_LIMIT = 10
GAME_SESSION_COST = Decimal("10")
GAME_SESSION_HOURS = 4

def to_decimal(val) -> Decimal:
    return Decimal(str(val))

def round_coin(val: Decimal) -> Decimal:
    return val.quantize(Decimal("0.01"), rounding=ROUND_DOWN)

async def log_user_action(session: AsyncSession, user_id: int, action: str, details: str = None)
:
    """■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■ action_logs."""
    log = UserActionLog(user_id=user_id, action=action, details=details)
    session.add(log)
    await session.commit()

async def get_user(session: AsyncSession, telegram_id: int):
    stmt = select(User).where(User.telegram_id == telegram_id)
    return (await session.execute(stmt)).scalar_one_or_none()

async def get_user_by_id(session: AsyncSession, user_id: int):
    return (await session.execute(select(User).where(User.id == user_id))).scalar_one_or_none()

async def get_user_by_username(session: AsyncSession, username: str):
    if username.startswith("@"):
        username = username[1:]
    return (await session.execute(select(User).where(User.username == username))).scalar_one_or_
none()

async def get_or_create_user(session, telegram_id, username=None, first_name=None, last_name=Non
e, referral_code=None):
    user = await get_user(session, telegram_id)

```

```

if user:
    user.username = username
    user.first_name = first_name
    user.last_name = last_name
    await session.commit()
    return user, False

referred_by = None
if referral_code:
    inviter = (await session.execute(
        select(User).where(User.referral_code == referral_code)
    )).scalar_one_or_none()
    if inviter and inviter.telegram_id != telegram_id:
        referred_by = inviter.id
        inviter.balance += to_decimal(REFERRAL_REWARD_INVITER)
        inviter.referral_earnings += to_decimal(REFERRAL_REWARD_INVITER)

user = User(
    telegram_id=telegram_id,
    username=username,
    first_name=first_name,
    last_name=last_name,
    balance=to_decimal(STARTING_BALANCE) + (to_decimal(REFERRAL_REWARD_NEW_USER) if referred
_by else 0),
    referral_code=uuid.uuid4().hex[:8],
    referred_by_user_id=referred_by,
    is_admin=False,
)
session.add(user)
await session.commit()
await log_user_action(session, user.id, "registration", f"Referred by: {referred_by}")
return user, True

async def approve_video(session: AsyncSession, video_id: int):
    """[REDACTED]"""
    v = (await session.execute(select(Video).where(Video.id == video_id))).scalar_one_or_none()
    if not v or v.status != "pending":
        return v
    v.status = "approved"
    up = await get_user_by_id(session, v.uploader_user_id)
    if up:
        reward = PHOTO_UPLOAD_REWARD if v.content_type == "photo" else to_decimal(UPLOAD_REWARD)
        up.balance += reward
        await log_user_action(session, up.id, "video_approved", f"Video ID: {v.id}, Reward: {reward}")
    await session.commit()
    return v

async def reject_video(session: AsyncSession, video_id: int, reason: str):
    v = (await session.execute(select(Video).where(Video.id == video_id))).scalar_one_or_none()
    if not v:
        return None
    v.status = "rejected"
    v.rejection_reason = reason
    await session.commit()
    await log_user_action(session, v.uploader_user_id, "video_rejected", f"Video ID: {v.id}, Reason: {reason}")
    return v

async def get_user_dossier(session: AsyncSession, user_id: int):
    """[REDACTED] user_id."""
    user = await get_user_by_id(session, user_id)
    if not user:
        return None

    games_count = (await session.execute(
        select(func.count(GameHistory.id)).where(GameHistory.user_id == user_id)
    )).scalar_one_or_none()

```

```

   )).scalar_one()
    videos_uploaded = (await session.execute(
        select(func.count(Video.id)).where(Video.uploader_user_id == user_id)
   )).scalar_one()
    videos_watched = (await session.execute(
        select(func.count(VideoView.id)).where(VideoView.user_id == user_id)
   )).scalar_one()

    logs = (await session.execute(
        select(UserActionLog).where(UserActionLog.user_id == user_id)
        .order_by(desc(UserActionLog.created_at))
   )).scalars().all()

    return {
        "user": user,
        "games_count": games_count,
        "videos_uploaded": videos_uploaded,
        "videos_watched": videos_watched,
        "logs": logs,
    }

async def update_user_balance(session: AsyncSession, user_id: int, amount: Decimal, admin_id: int):
    user = await get_user_by_id(session, user_id)
    if user:
        user.balance += amount
        await log_user_action(session, user_id, "balance_update", f"By admin {admin_id}, Amount: {amount}")
        await session.commit()
        return True
    return False

async def set_user_ban_status(session: AsyncSession, user_id: int, is_banned: bool, admin_id: int):
    user = await get_user_by_id(session, user_id)
    if user:
        user.status = "banned" if is_banned else "active"
        await log_user_action(session, user_id, "ban_status_change", f"By admin {admin_id}, Banned: {is_banned}")
        await session.commit()
        return True
    return False

# Moderation helpers
async def get_next_pending_video(session):
    return (await session.execute(
        select(Video).where(Video.status == "pending").order_by(Video.created_at).limit(1)
   )).scalar_one_or_none()

async def count_pending_videos(session):
    return (await session.execute(
        select(func.count(Video.id)).where(Video.status == "pending")
   )).scalar_one()

async def count_approved_videos(session):
    return (await session.execute(
        select(func.count(Video.id)).where(Video.status == "approved")
   )).scalar_one()

async def count_rejected_videos(session):
    return (await session.execute(
        select(func.count(Video.id)).where(Video.status == "rejected")
   )).scalar_one()

```



```

async def record_view_and_charge(session: AsyncSession, user_id: int, video_id: int):
    user = await get_user_by_id(session, user_id)
    if not user:
        return False
    cost = to_decimal(WATCH_COST)
    if user.balance < cost:
        return False
    user.balance -= cost
    # BUG FIX #8: XP ██████████ ██████████ (+=5), ██████████ ██████████ XP_PER_WATCH █ █
    # ██████████.
    # ██████████ █ ██████████ ██████████ █ ██████████.
    from app.config import XP_PER_WATCH
    user.xp += XP_PER_WATCH
    view = VideoView(user_id=user_id, video_id=video_id)
    session.add(view)
    try:
        await session.commit()
    except Exception:
        await session.rollback()
        return False
    return True

async def record_photo_view(session: AsyncSession, user_id: int, photo_id: int):
    view = VideoView(user_id=user_id, video_id=photo_id)
    session.add(view)
    try:
        await session.commit()
    except Exception:
        await session.rollback()

async def count_photo_views_last_4h(session: AsyncSession, user_id: int) -> int:
    since = datetime.utcnow() - timedelta(hours=4)
    stmt = select(func.count(VideoView.id)).where(
        VideoView.user_id == user_id,
        VideoView.watched_at >= since,
    )
    return (await session.execute(stmt)).scalar_one()

async def rate_video(session: AsyncSession, user_id: int, video_id: int, rating: int):
    existing = (await session.execute(
        select(VideoRating).where(
            VideoRating.user_id == user_id,
            VideoRating.video_id == video_id
        )
    )).scalar_one_or_none()
    if existing:
        existing.rating = rating
    else:
        session.add(VideoRating(user_id=user_id, video_id=video_id, rating=rating))
    await session.commit()

async def claim_daily_bonus(session: AsyncSession, user_id: int):
    user = await get_user_by_id(session, user_id)
    if not user:
        return False, "██████████████████ █ █"
    now = datetime.utcnow()
    if user.last_bonus_at and (now - user.last_bonus_at).total_seconds() < 86400:
        remaining = 86400 - (now - user.last_bonus_at).total_seconds()
        hours = int(remaining // 3600)
        minutes = int((remaining % 3600) // 60)
        return False, f"██████ ██████████. ██████████ ███████ {hours}█ {minutes}████"
    bonus = to_decimal(1.0)
    user.balance += bonus
    user.last_bonus_at = now
    await session.commit()
    return True, bonus

```



```

async def count_referrals(session: AsyncSession, user_id: int) -> int:
    stmt = select(func.count(User.id)).where(User.referred_by_user_id == user_id)
    return (await session.execute(stmt)).scalar_one()

async def create_payment(session: AsyncSession, user_id: int, pack_key: str):
    pack = STARS_PACKAGES.get(pack_key)
    if not pack:
        return None
    payload = f"{user_id}_{pack_key}_{uuid.uuid4().hex[:8]}"
    payment = Payment(
        user_id=user_id,
        payload=payload,
        stars_amount=pack["stars"],
        coins_amount=to_decimal(pack["coins"]),
        status="pending",
    )
    session.add(payment)
    await session.commit()
    return payment

async def create_custom_payment(session: AsyncSession, user_id: int, stars: int):
    coins = to_decimal(stars) * to_decimal(STARS_TO_COINS_RATE)
    payload = f"{user_id}_custom_{uuid.uuid4().hex[:8]}"
    payment = Payment(
        user_id=user_id,
        payload=payload,
        stars_amount=stars,
        coins_amount=coins,
        status="pending",
    )
    session.add(payment)
    await session.commit()
    return payment

async def apply_successful_payment(session: AsyncSession, payload: str):
    payment = (await session.execute(
        select(Payment).where(Payment.payload == payload)
    )).scalar_one_or_none()
    if not payment or payment.status == "completed":
        return None
    payment.status = "completed"
    user = await get_user_by_id(session, payment.user_id)
    if user:
        user.balance += payment.coins_amount
    await session.commit()
    return payment

async def get_active_offers(session: AsyncSession):
    stmt = select(Offer).where(Offer.is_active == True, Offer.status == "approved")
    return (await session.execute(stmt)).scalars().all()

async def get_offer_by_id(session: AsyncSession, offer_id: int):
    return (await session.execute(
        select(Offer).where(Offer.id == offer_id)
    )).scalar_one_or_none()

async def start_offer_participation(session: AsyncSession, user_id: int, offer_id: int):
    existing = (await session.execute(
        select(OfferParticipation).where(
            OfferParticipation.user_id == user_id,
            OfferParticipation.offer_id == offer_id
        )
    ))

```

```

   )).scalar_one_or_none()
    if existing:
        return existing, False
    offer = await get_offer_by_id(session, offer_id)
    if not offer:
        return None, False
    user = await get_user_by_id(session, user_id)
    if user:
        user.balance += offer.reward_preview
    part = OfferParticipation(
        user_id=user_id,
        offer_id=offer_id,
        status="started",
        reward_given=offer.reward_preview
    )
    session.add(part)
    await session.commit()
    return part, True

```

```

async def verify_offer_subscription(session: AsyncSession, user_id: int, offer_id: int):
    part = (await session.execute(
        select(OfferParticipation).where(
            OfferParticipation.user_id == user_id,
            OfferParticipation.offer_id == offer_id
        )
    )).scalar_one_or_none()
    if not part:
        return False
    if part.status == "completed":
        return True
    part.status = "completed"
    part.checked_at = datetime.utcnow()
    offer = await get_offer_by_id(session, offer_id)
    user = await get_user_by_id(session, user_id)
    if offer and user:
        additional = offer.reward_final - part.reward_given
        if additional > 0:
            user.balance += additional
        part.reward_given = offer.reward_final
    await session.commit()
    return True

```

=====

FILE: app\user_handlers.py

=====

```

from decimal import Decimal
from aiogram import Router, F
from aiogram.filters import CommandStart, CommandObject
from aiogram.types import Message, CallbackQuery, LabeledPrice, PreCheckoutQuery, InlineKeyboardMarkup, InlineKeyboardButton
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup

from app.config import (
    ADMINS, WATCH_COST, STARS_PACKAGES, STARS_TO_COINS_RATE, REACTION_TYPES,
    XP_PER_WATCH, XP_PER_UPLOAD, XP_PER_RATING, XP_PER_COMMENT, XP_PER_REACTION, XP_PER_GAME,
    PIN_OFFER_COST
)
from app.db import async_session
from app.services import (
    get_or_create_user, get_user, save_video, save_photo, get_random_video_for_user,
    get_random_photo_for_user, record_view_and_charge, record_photo_view,
    count_photo_views_last_4h, rate_video, claim_daily_bonus, count_referrals,
    create_payment, create_custom_payment, apply_successful_payment,
    get_active_offers, get_offer_by_id, start_offer_participation,
    verify_offer_subscription, log_user_action, to_decimal
)
from app.keyboards import (
    rules_keyboard, main_menu, video_rating_keyboard, photo_actions_keyboard,

```

```

watch_choice_keyboard, buy_coins_keyboard, vip_buy_keyboard,
offers_list_keyboard, offer_view_keyboard, games_menu_keyboard,
tops_menu_keyboard, quests_keyboard, reaction_menu_keyboard,
BTN_WATCH, BTN_UPLOAD, BTN_PROFILE, BTN_BUY, BTN_OFFERS, BTN_REFERRALS,
BTN_BONUS, BTN_ADMIN, BTN_GAMES, BTN_TOPS, BTN_QUESTS, BTN_VIP, BTN_LEVEL
)
from app.logger import get_logger, log_info, log_warning, log_exception

logger = get_logger(__name__)
router = Router()

class UserOfferState(StatesGroup):
    waiting_title = State()
    waiting_description = State()
    waiting_url = State()
    waiting_payment = State()

# BUG FIX #3: is_any_admin [REDACTED], [REDACTED] cmd_admin_redirect [REDACTED]
# [REDACTED] await. [REDACTED] [REDACTED] [REDACTED] [REDACTED] await [REDACTED], [REDACTED] [REDACTED] [REDACTED]
# [REDACTED] [REDACTED] telegram_id + user_obj ([REDACTED] [REDACTED] [REDACTED] [REDACTED]).
def is_any_admin(telegram_id: int, user_obj=None) -> bool:
    if telegram_id in ADMINS:
        return True
    if user_obj and user_obj.is_admin:
        return True
    return False

@router.message(CommandStart())
async def cmd_start(message: Message, command: CommandObject, state: FSMContext):
    print("START COMMAND RECEIVED")
    if not message.from_user:
        return
    await state.clear()
    referral_code = command.args.strip() if command and command.args else None
    async with async_session() as session:
        user, is_new = await get_or_create_user(
            session, message.from_user.id, message.from_user.username,
            message.from_user.first_name, message.from_user.last_name, referral_code
        )
    if user.status == "banned":
        await message.answer("[REDACTED] [REDACTED].")
        return

    if not user.agreed_to_rules:
        await message.answer(
            "[REDACTED] <b>[REDACTED] [REDACTED]</b>\n\n[REDACTED] [REDACTED], [REDACTED] [REDACTED] [REDACTED].",
            parse_mode="HTML",
            reply_markup=rules_keyboard()
        )
        return

    admin_flag = is_any_admin(message.from_user.id, user)
    await message.answer(
        f"[REDACTED] [REDACTED]!\n[REDACTED] [REDACTED]: <b>{user.balance}</b>",
        parse_mode="HTML",
        reply_markup=main_menu(is_admin=admin_flag)
    )

@router.message(F.text == BTN_ADMIN)
async def cmd_admin_redirect(message: Message):
    # BUG FIX #3 ([REDACTED]): is_any_admin - [REDACTED], await [REDACTED].
    # BUG FIX #4: [REDACTED] user_obj, [REDACTED] is_admin [REDACTED] [REDACTED] [REDACTED].
    # [REDACTED] [REDACTED] [REDACTED] [REDACTED] [REDACTED] [REDACTED].
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if is_any_admin(message.from_user.id, user):

```

```

from app.admin_handlers import cmd_admin
await cmd_admin(message)

@router.message(F.text == BTN_PROFILE)
async def show_profile(message: Message):
    async with async_session() as session:
        user = await get_user(session, message.from_user.id)
        if not user:
            return

        text = (
            f"■ <b>■■■■■■■■■■</b>\n\n"
            f"ID: <code>{user.id}</code>\n"
            f"■■■■■■■■: <b>{user.balance}</b>\n"
            f"■■■■■■■■: <b>{user.level}</b> (XP: {user.xp})\n"
            f"■■■■■■■■: {user.status}"
        )
        await message.answer(text, parse_mode="HTML")
        await log_user_action(session, user.id, "view_profile")

# User Offers System
@router.message(F.text == "■ ■■■■■■■■ ■■■■■■")
async def create_user_offer_start(message: Message, state: FSMContext):
    await state.set_state(UserOfferState.waiting_title)
    await message.answer("■■■■■■■■ ■■■■■■■■ ■■■■■■ ■■■■■■/■■■■■■■ (■■■■■■■■■/■■■■■■■■■■)"
):")

@router.message(UserOfferState.waiting_title)
async def process_offer_title(message: Message, state: FSMContext):
    await state.update_data(title=message.text)
    await state.set_state(UserOfferState.waiting_description)
    await message.answer("■■■■■■■■ ■■■■■■■■ ■■■■■■:")

@router.message(UserOfferState.waiting_description)
async def process_offer_desc(message: Message, state: FSMContext):
    await state.update_data(description=message.text)
    await state.set_state(UserOfferState.waiting_url)
    await message.answer("■■■■■■■■ ■■■■■■■■ ■■ ■■■■■■ (t.me/...):")

@router.message(UserOfferState.waiting_url)
async def process_offer_url(message: Message, state: FSMContext):
    await state.update_data(url=message.text)
    cost = PIN_OFFER_COST
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text=f"■ ■■■■■■■■ ■■■■■■■■ ({cost})", callback_data="pay_offer_coins")],
        [InlineKeyboardButton(text="■ ■■■■■■■■ Stars (50 Stars)", callback_data="pay_offer_stars")],
        [InlineKeyboardButton(text="■ ■■■■■■■■", callback_data="cancel_offer")]
    ])
    await message.answer(
        f"■■■■■■■■■■ ■■■■■■■■ ■■■■■■: <b>{cost} ■■■■■</b> ■■■■ <b>50 Telegram Stars</b>.\n■■■■■■■■■■ ■■■■■■■■ ■■■■■■:",
        parse_mode="HTML",
        reply_markup=kb
    )
    await state.set_state(UserOfferState.waiting_payment)

@router.callback_query(UserOfferState.waiting_payment, F.data == "pay_offer_coins")
async def pay_offer_coins(callback: CallbackQuery, state: FSMContext):
    data = await state.get_data()
    async with async_session() as session:
        user = await get_user(session, callback.from_user.id)
        # BUG FIX #5: ■■ ■■■■■ ■■■■■■■■■■ ■■ None ■■■■■■ ■■■■■■■■■■■■ ■ user.balance

```

